

Software Design Specification (SDS) Report

IntelliFly : Flight Scheduler System

Project Team

Student 1 24K-0579

Student 2 24K-0855

Student 3 24K-0721

Session 2024-2028

Course Instructor

Sir Syed Ahmed Khan



**Department of Computer Science
National University of Computer and Emerging Sciences
Karachi, Pakistan
May, 2026**

Contents

Chapter 1: Introduction.....	3
System Overview.....	3
Problem Statement.....	3
Scope	3
Objectives.....	4
Users and Stakeholders	4
System Requirements Summary	4
Software Process Model.....	5
Chapter 2: System Analysis and Modeling.....	6
Use Case Diagram	6
Use Case Description	7
Class Diagram	8
Activity Diagrams.....	9
Sequence Diagrams	10
Component Diagram.....	11
Deployment Diagram	12
Chapter 3: Architectural View (4+1 Model).....	13
Logical View	13
Process View	13
Development View.....	14
Physical View	15
Use Case View	15
References.....	16

Chapter 1: Introduction

System Overview

IntelliFly is a proposed console-based flight scheduling and airport management system designed to handle complex operations at busy airports. The system will be built using C++ and will manage flight scheduling, runway allocation, route planning, and efficient searching across airports and flights. The design focuses on optimizing performance using advanced data structures and algorithms to handle real-time scheduling decisions and large volumes of flight data.

The system aims to provide airport staff with tools to manage daily operations more efficiently by automating tasks like flight priority assignment, runway conflict resolution, and route optimization between airports.

Problem Statement

Airport operations involve managing hundreds of flights daily with strict time constraints and limited resources like runways. Current manual or basic systems struggle with priority-based scheduling, efficient search, and dynamic route planning. Delays in finding available runways or optimal routes can cascade into larger operational disruptions.

IntelliFly addresses these problems by designing a system that uses efficient data structures for quick lookups, intelligent scheduling algorithms for priority management, and graph-based routing to find optimal paths between airports.

Scope

The system will include the following features:

- Flight management operations such as adding new flights, canceling existing ones, and viewing schedules
- Runway allocation that prevents conflicts and assigns runways based on availability and priority
- Priority-based flight scheduling using a heap structure
- Route planning between airports using graph algorithms
- Airport name autocomplete to help users quickly find airports
- Advanced flight search with pattern matching algorithms
- Sorting functionality for organizing flight data
- Undo functionality to reverse recent actions

The system will be limited to console interaction and will not include graphical interfaces, real-time external API integration, or multi-user concurrent access in the initial design phase.

Objectives

The main objectives of IntelliFly are:

- Design a system that efficiently manages flight scheduling with minimal delays
- Implement fast search and retrieval mechanisms for flights and airports
- Provide intelligent runway allocation to prevent conflicts
- Enable route optimization between multiple airports
- Create a user-friendly console interface for airport staff
- Ensure the system can handle large datasets without performance degradation
- Allow easy modifications and rollback through undo functionality

Users and Stakeholders

The primary users of IntelliFly will be airport operations staff including flight coordinators, air traffic controllers, and airport managers. These users will interact with the system to manage daily flight operations, allocate runways, and plan routes.

Stakeholders include airport management who need operational efficiency, airlines that depend on timely scheduling, and passengers who benefit indirectly from reduced delays and better resource management.

System Requirements Summary

Functional Requirements:

- The system must allow users to add, cancel, and view flights
- The system must allocate runways based on availability and priority
- The system must schedule flights using priority queues
- The system must find shortest paths and minimum spanning trees for route planning
- The system must support autocomplete for airport names
- The system must provide advanced search using pattern matching
- The system must support undo operations for recent changes

Non-Functional Requirements:

- The system should respond to queries within acceptable time limits even with large datasets
- The system should be reliable and handle edge cases like duplicate entries or invalid data
- The code should be modular to allow future enhancements
- The system should provide clear error messages and guidance to users

Technical Requirements:

- The system will be developed in C++
- AVL trees will be used for indexed scheduling
- B-trees will handle flight indexing for efficient range queries

- Tries will enable fast autocomplete functionality
- Graphs will represent airport networks
- Heaps will manage priority-based scheduling
- Hash tables will provide constant-time lookups where needed

Software Process Model

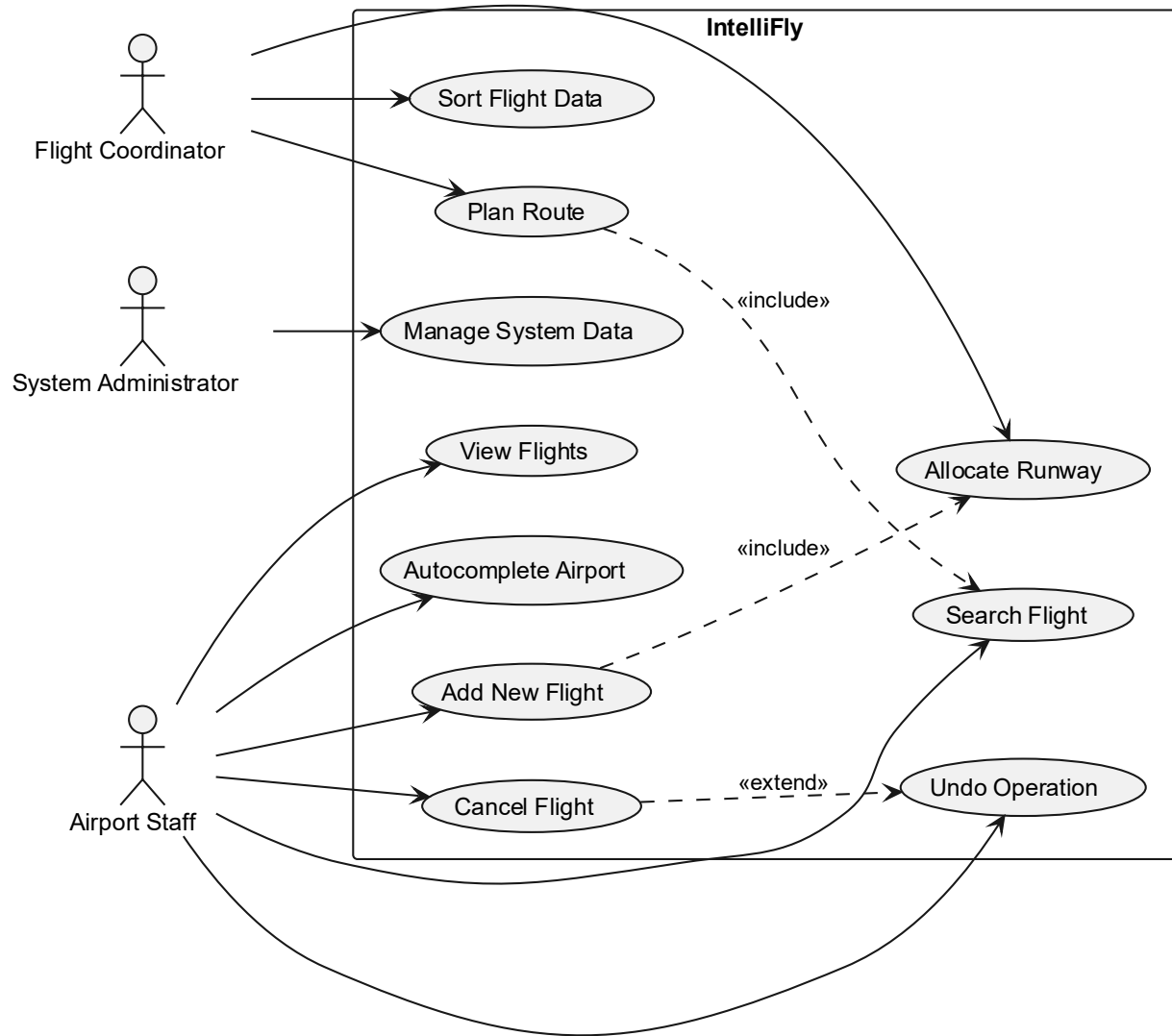
The development of IntelliFly will follow an iterative model. This approach is suitable because the system involves multiple complex components like data structures, algorithms, and user interaction that need to be developed and tested incrementally.

Each iteration will focus on implementing and testing a specific module such as flight management, runway allocation, or route planning. This allows us to identify design issues early and refine the system based on testing feedback before moving to the next component. The iterative model also gives us flexibility to adjust our design if we encounter unexpected challenges with algorithm performance or data structure selection.

We chose this over a pure waterfall model because we expect to learn more about performance requirements as we implement and test each component. An agile approach would be too informal for an academic project that requires clear documentation and structured design phases.

Chapter 2: System Analysis and Modeling

Use Case Diagram

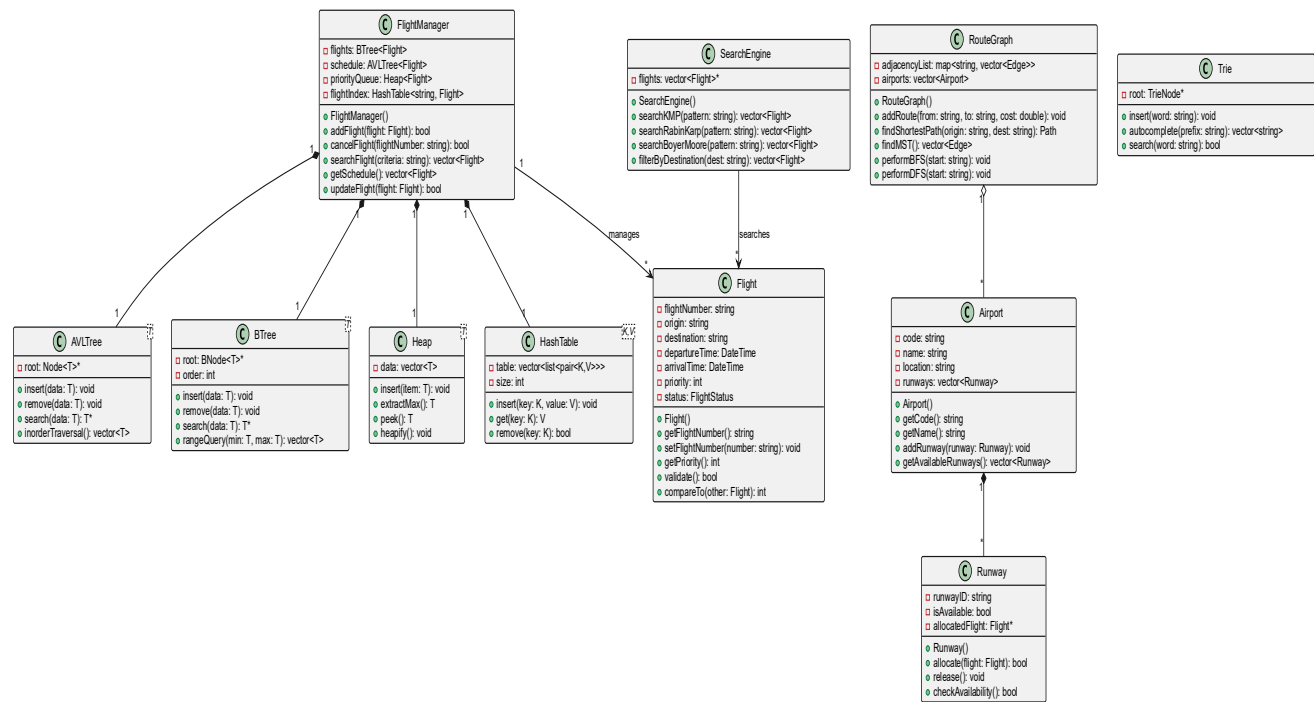


The use case diagram shows the main actors (Airport Staff, Flight Coordinator, System Administrator) and their interactions with the system including managing flights, allocating runways, searching for flights, planning routes, and using autocomplete features.

Use Case Description

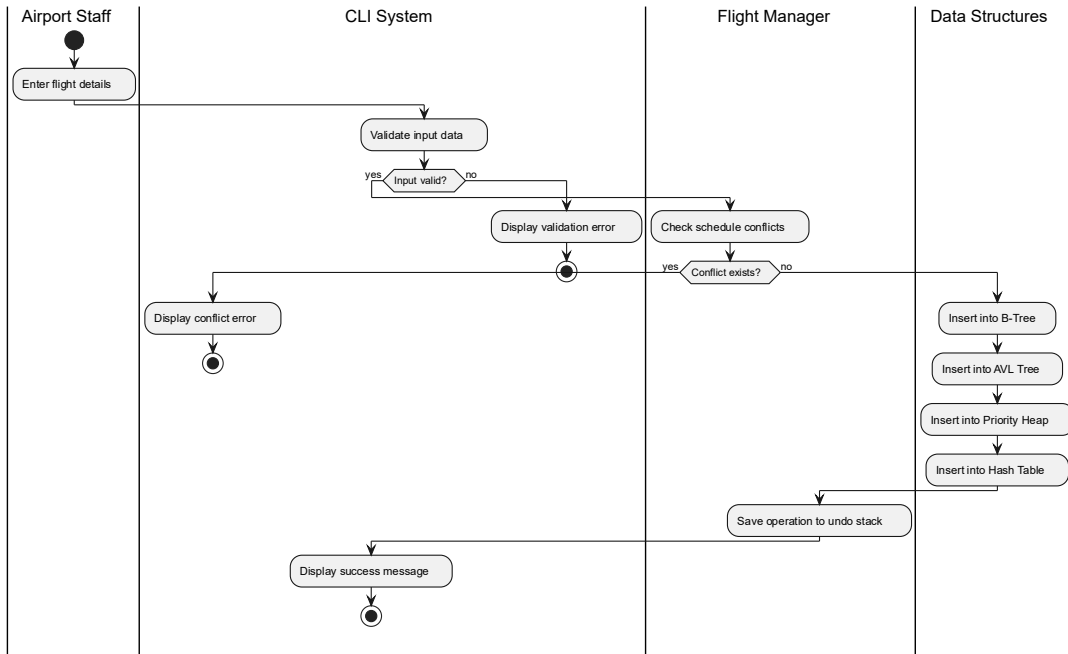
Use Case	Description
Add New Flight	The user enters flight information such as flight number, route, departure time, and priority. The system validates the data, checks scheduling conflicts, and stores the flight in the required data structures.
Allocate Runway	The user requests runway allocation for a flight. The system checks runway availability and flight priority, then assigns a suitable runway or suggests the next available slot.
Search Flight	The user searches using flight number, destination, or keywords. The system applies search algorithms to find and display matching flight records.
Plan Route	The user enters origin and destination airports. The system uses graph algorithms to calculate and display the shortest or most suitable route.
Autocomplete Airport Name	As the user types an airport code or name, the system provides suggestions using the trie data structure.
Cancel Flight	The user selects a flight to cancel. The system removes the flight from scheduling structures, frees allocated resources, and stores the action for undo support.
Undo Operation	The user requests to reverse the most recent action. The system restores the previous state using the undo stack mechanism.

Class Diagram

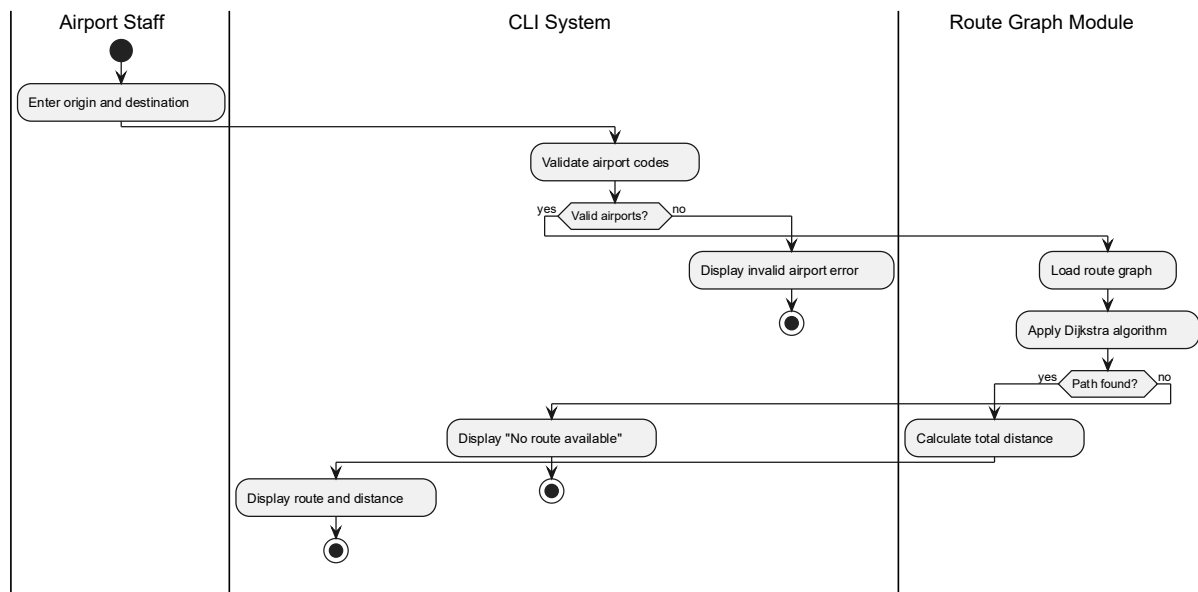


The class diagram shows the main classes in the system including Flight, Airport, Runway, FlightManager, RouteGraph, SearchEngine, and various data structure classes like AVLTree, BTree, Trie, Heap, and HashTable. Relationships such as composition, aggregation, and inheritance will be indicated where applicable.

Activity Diagrams

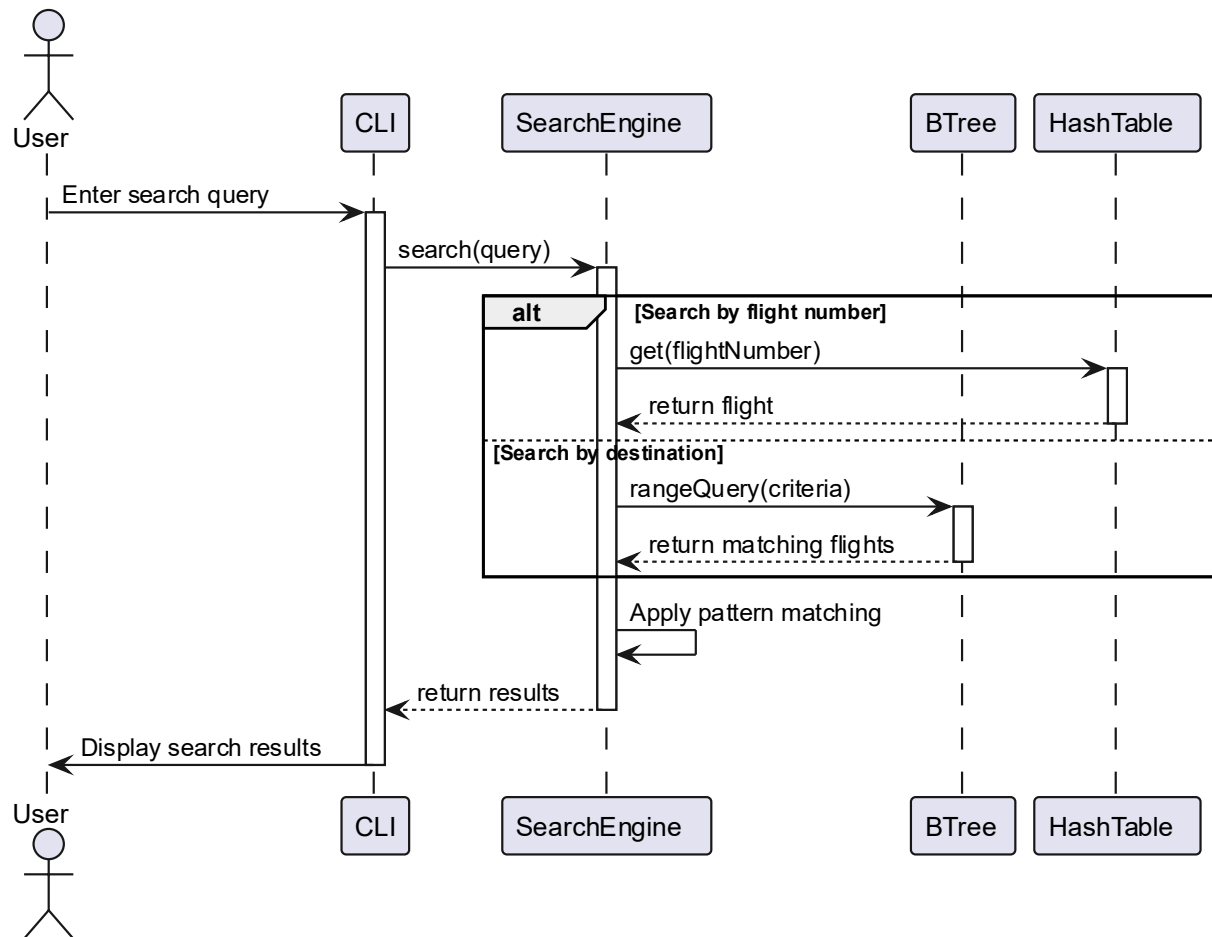


The activity diagram for adding a flight shows the flow from user input, through validation steps, conflict checking, insertion into multiple data structures, and final confirmation or error handling.

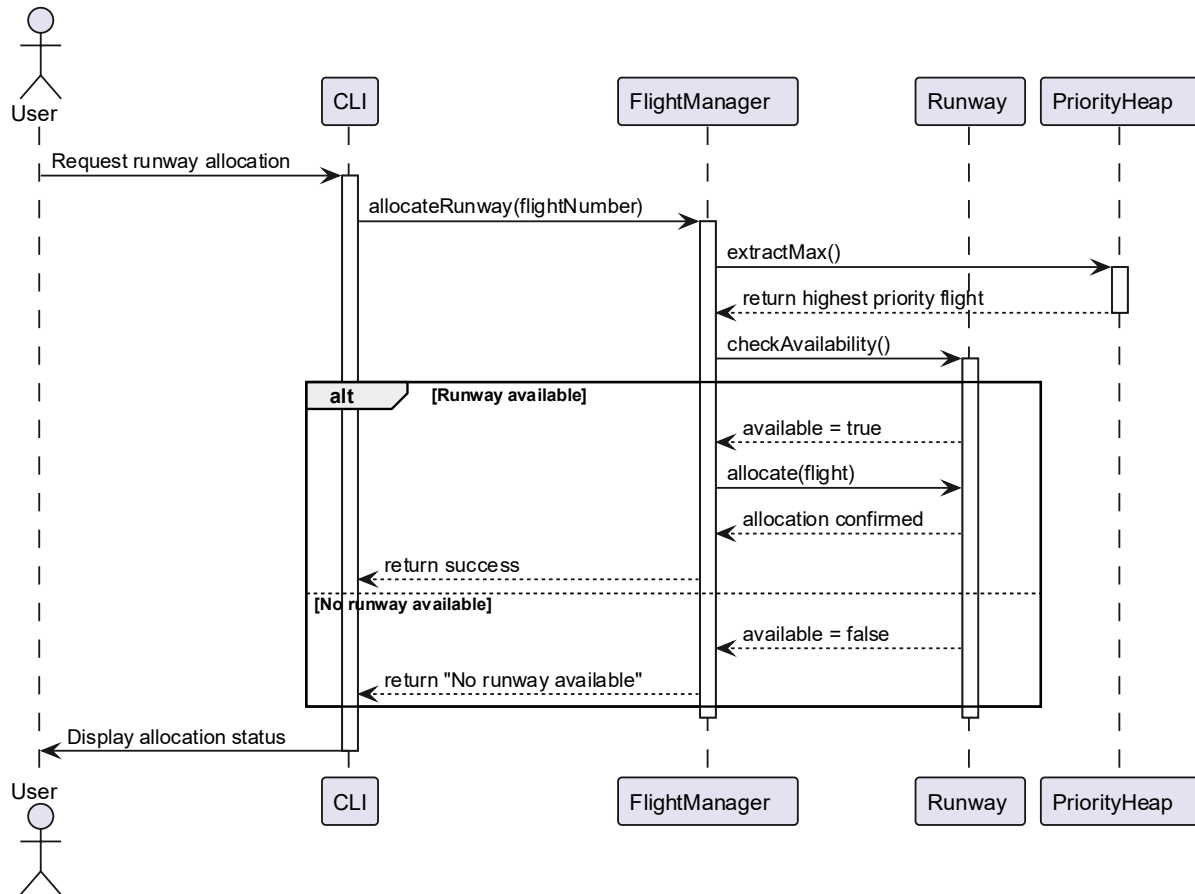


The activity diagram for route planning illustrates the process of receiving origin and destination, constructing or accessing the graph, applying the pathfinding algorithm, and presenting the results to the user.

Sequence Diagrams

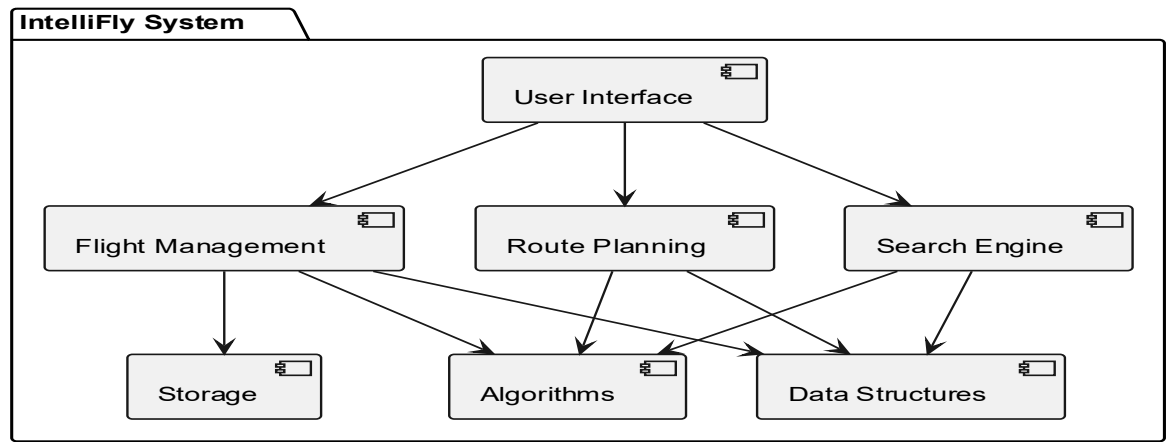


The sequence diagram for flight search shows interactions between the User, CLI interface, SearchEngine, and relevant data structures (B-tree or hash table) to retrieve and display search results.



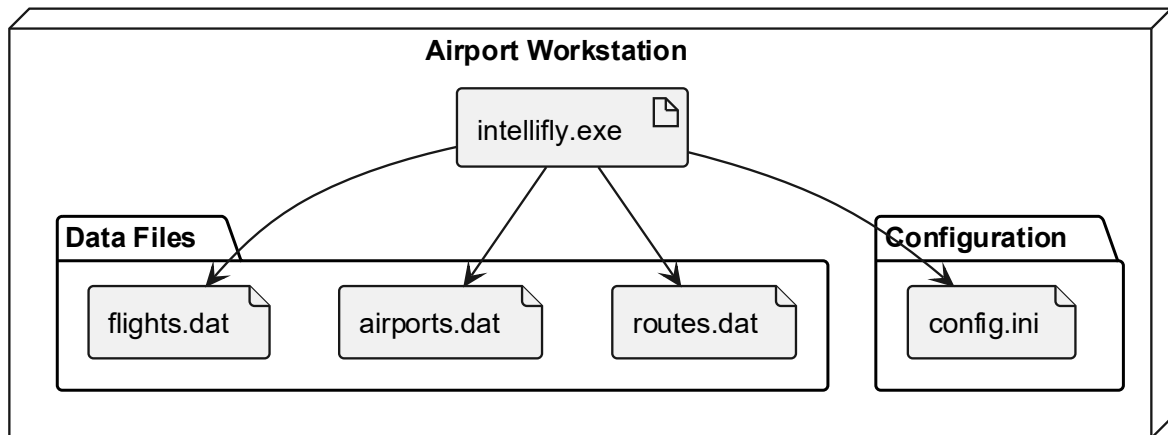
The sequence diagram for runway allocation demonstrates the message flow between User, FlightManager, RunwayController, and data structures when allocating a runway to a flight based on availability and priority.

Component Diagram



The component diagram shows the high level organization of the system into components such as the User Interface component, Flight Management component, Data Structures component, Algorithms component, and Storage component. Dependencies between components will be indicated.

Deployment Diagram



The deployment diagram illustrates the physical deployment of the system on a single machine running the console application. It will show the executable, associated libraries, and data files stored on the local file system.

Timing diagrams were not included in this design because the system primarily focuses on structural organization and process interaction rather than strict real-time synchronization behavior.

Chapter 3: Architectural View (4+1 Model)

Logical View

The logical view describes the functionality that the system provides to users through its main modules and classes.

The core of the system revolves around the Flight entity, which contains attributes like flight number, origin, destination, departure time, arrival time, and priority. The FlightManager class handles all operations related to flights including adding, removing, searching, and scheduling.

The Runway class represents physical runways with attributes for runway ID, availability status, and allocated flights. The RunwayController manages runway assignments and prevents conflicts.

The RouteGraph class represents the network of airports as a graph where nodes are airports and edges represent routes with associated distances or costs. This class provides methods to find shortest paths and minimum spanning trees.

The SearchEngine class encapsulates different pattern matching algorithms and provides a unified interface for searching flight information based on various criteria.

Data structure classes (AVLTree, BTree, Trie, Heap, HashTable) are designed as template classes or specialized implementations that support the operations needed by the higher-level managers. These structures are encapsulated to hide implementation details from the main business logic.

Process View

The process view addresses the runtime behavior and concurrency aspects of the system.

Since IntelliFly is designed as a single-user console application, there will not be multiple concurrent processes in the initial design. However, the system will be structured to handle operations in a logical sequence to maintain data consistency.

When a user initiates an operation like adding a flight, the process follows these steps: validate input, check for conflicts by querying existing data structures, update all relevant data structures (B-tree, AVL tree, heap, hash table) in the correct order, and commit the changes. If any step fails, the system should roll back partial changes to maintain consistency.

The undo mechanism maintains a command history stack that stores reversible operations. When undo is invoked, the system pops the last command and executes its reverse operation.

Priority scheduling operates by maintaining a heap structure where flights are ordered by priority and time. When a runway becomes available, the system extracts the highest priority flight from the heap and allocates it.

Development View

The development view describes how the system will be organized from a programmer's perspective.

The codebase will be structured into multiple header and implementation file pairs:

Flight.h / Flight.cpp will define the Flight class with attributes and basic operations like setting and getting flight details, validating data, and comparing flights.

Airport.h / Airport.cpp will handle airport entities with information like airport code, name, location, and associated facilities.

Runway.h / Runway.cpp will manage runway entities including availability checking and allocation tracking.

FlightManager.h / FlightManager.cpp will contain the core scheduling logic, flight operations, and coordination between different data structures. This module acts as the main controller for flight-related functionality.

RouteGraph.h / RouteGraph.cpp will implement the graph data structure and routing algorithms including Dijkstra's algorithm for shortest paths and Prim's/Kruskal's for minimum spanning trees. It will also include BFS and DFS for graph traversal.

SearchEngine.h / SearchEngine.cpp will implement pattern matching algorithms (KMP, Rabin-Karp, Boyer-Moore) and provide search functionality across flight records.

DataStructures/ will be a directory containing modular implementations:

- AVLTree.h for self-balancing binary search tree used in schedule indexing
- BTree.h for B-tree implementation used in flight indexing
- Trie.h for prefix tree used in autocomplete
- Heap.h for priority queue implementation
- HashTable.h for fast key-value lookups

CLI.cpp will handle all user interface logic including displaying menus, reading input, validating commands, calling appropriate manager functions, and displaying results or error messages.

Utils.h / Utils.cpp will contain utility functions like sorting algorithms (merge sort, quick sort), input validation helpers, and common operations used across modules.

main.cpp will serve as the entry point, initializing the system, loading any persisted data, and starting the main interaction loop.

This modular structure allows different team members to work on separate components and makes testing and debugging easier. Each module will have clear interfaces and minimal dependencies on other modules.

Physical View

The physical view describes how the system will be deployed on hardware.

IntelliFly will run as a standalone console application on a standard desktop or laptop computer. The system will be compiled into a single executable file that users can run from a command line interface.

The deployment will consist of:

- The main executable (intellifly.exe or intellifly on Unix systems)
- Data files stored locally to persist flight information, airport data, and routes between sessions
- Configuration files if needed for system parameters

The system does not require network connectivity, web servers, or database servers in its initial design. All data will be managed in-memory during runtime and saved to local files when the user exits the application.

For a production environment, the system could be deployed on airport workstations with adequate processing power and memory to handle the expected volume of flights. The system requirements would be minimal by modern standards since it operates entirely through console I/O without graphics.

Use Case View

The use case view ties together the other views by showing how they support the main use cases described earlier.

For the "Add New Flight" use case, the logical view provides the Flight and FlightManager classes, the process view ensures data consistency across updates, the development view shows how Flight.cpp and FlightManager.cpp implement the logic, and the physical view confirms the operation runs locally on the user's machine.

For "Plan Route," the logical view provides RouteGraph and algorithm implementations, the process view handles the computation steps, the development view shows RouteGraph.cpp contains Dijkstra's and MST algorithms, and the physical view indicates the graph data is maintained in memory.

For "Search Flight," the logical view provides SearchEngine, the process view describes the search flow, the development view shows SearchEngine.cpp implements pattern matching algorithms, and the physical view confirms quick response times on local data.

This view demonstrates how the architecture supports all functional requirements through coordinated interaction of classes, processes, modules, and deployment.

References

- UML 2 Toolkit by Hans-Erik Eriksson, Magnus Penker. Brian Lyons, David Fado
- UML Distilled, Third Edition: A Brief Guide to the Standard Object Modeling Language by Martin Fowler
- Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, Third Edition By Craig Larman
- <https://plantuml.com/use-case-diagram>
- <https://plantuml.com/class-diagram>
- <https://plantuml.com/activity-diagram-beta>
- <https://plantuml.com/component-diagram>
- <https://plantuml.com/deployment-diagram>